

Clase 015 — Python para seguridad: fundamentos del lenguaje

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Seitz & Arnold, Black Hat Python (2ª ed.)* 🕒

Duración estimada: **120 min** · Nivel: **Fundamentos**

Objetivo

Adquirir la base de Python necesaria para escribir herramientas de seguridad: tipos, estructuras de datos, control de flujo, funciones, manejo de archivos y excepciones. Python es el lenguaje franco de la ciberseguridad ofensiva y defensiva, y esta clase asienta lo que usarás en sockets y Scapy.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Manejar** tipos, listas, diccionarios y comprensiones.
2. **Escribir** control de flujo, funciones y módulos.
3. **Leer y escribir** archivos y procesar su contenido.
4. **Gestionar** errores con excepciones y usar la `stdlib`.
5. **Crear** un script CLI de utilidad para seguridad.

Temas

#	Tema	Por qué importa
1	Tipos y variables	<code>str</code> , <code>int</code> , <code>bytes</code> , <code>bool</code>
2	Estructuras de datos	<code>list</code> , <code>dict</code> , <code>set</code> , <code>tuple</code>
3	Control de flujo	<code>if</code> , <code>for</code> , <code>while</code> , comprensiones
4	Funciones y módulos	Reutilización y organización
5	Archivos	Leer logs, escribir informes
6	Excepciones	Robustez ante fallos
7	Librería estándar	<code>os</code> , <code>sys</code> , <code>subprocess</code> , <code>argparse</code> , <code>hashlib</code>
8	Entornos virtuales	Aislar dependencias con <code>venv</code> / <code>pip</code>

Definiciones y características

- **str vs. bytes**: texto (unicode) frente a datos binarios crudos. Clave: la red y la cripto trabajan con `bytes` ; conviértelos con `.encode()` / `.decode()` .

- **Diccionario:** mapa clave→valor. Clave: base para contar, indexar y estructurar resultados (p. ej. IP→puertos).
- **Comprensión de listas:** `[f(x) for x in it if cond]` . Clave: transforma y filtra en una línea.
- **Excepción:** mecanismo de control de errores con `try/except` . Clave: evita que un fallo de red tumbe todo el script.
- **argparse:** módulo estándar para CLIs. Clave: parsea argumentos con ayuda y validación gratis.
- **venv:** entorno virtual aislado. Clave: separa dependencias por proyecto y evita romper el Python del sistema.

Herramientas y preparación

Necesitas **Python 3** (ya en Kali) y un editor (VS Code recomendado con la extensión de Python). Crea un entorno virtual por proyecto:

```
python3 -m venv venv && source venv/bin/activate
pip install requests
```

Familiarízate con el REPL (`python3`) para experimentar y con `pip` para instalar librerías.

Laboratorio guiado

1. **REPL y tipos.** Explora conversiones str/bytes:

```
python b = "admin".encode(); print(b, b.hex(), b.decode())
```

2. **Estructuras de datos.** Cuenta ocurrencias de IPs en una lista con un diccionario, y luego con `collections.Counter` .
3. **Leer un log.** Escribe un script que abra un archivo de log y cuente líneas con "error":

```
python with open("app.log") as f: errores = sum(1 for line in f if "error" in line.lower())
print(f"Errores: {errores}")
```

4. **Funciones y módulos.** Extrae la lógica a una función `contar_patron(ruta, patron)` y llama desde `main()` .
5. **Excepciones.** Envuelve la apertura en `try/except FileNotFoundError` para fallar con gracia.
6. **CLI con argparse.** Convierte el script en una herramienta:

```
python import argparse p = argparse.ArgumentParser() p.add_argument("ruta"); p.add_argument("-patron", default="error") args = p.parse_args()
```

7. **hashlib.** Añade una función que calcule el SHA-256 de un archivo (adelanto de la Clase 021).

Ejercicios

1. Escribe una comprensión que devuelva solo las IPs privadas de una lista.
2. Crea un diccionario que agrupe usuarios por su shell leyendo `/etc/passwd` .
3. Implementa una función que valide si una cadena es una IPv4 bien formada (sin regex).
4. Lee un archivo grande línea a línea y extrae todas las que contengan un código HTTP 5xx.
5. Añade manejo de excepciones para permisos denegados y archivo inexistente.
6. Empaqueta tu utilidad con `argparse` , incluyendo `--help` y un argumento opcional.

Reto verificable

Escribe `logstats.py`, una herramienta CLI que reciba la ruta de un log y un patrón, y produzca un pequeño informe: total de líneas, líneas coincidentes, top 5 de las IPs más frecuentes (si las hay) y el SHA-256 del archivo. Debe manejar errores de archivo con mensajes claros y tener `--help`.

Criterio de aceptación: ejecutar `python3 logstats.py --help` muestra el uso; corriendo sobre un log real produce el informe correcto; y ante un archivo inexistente termina con un mensaje amable y código de salida $\neq 0$. El SHA-256 coincide con el de `sha256sum`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>TypeError: a bytes-like object is required</code>	Mezclas str y bytes. Convierte con <code>.encode()</code> / <code>.decode()</code> .
<code>UnicodeDecodeError</code> al leer un archivo	Codificación distinta. Abre con <code>encoding='utf-8'</code> , <code>errors='replace'</code> o en modo binario.
El script modifica el Python del sistema	No usaste venv. Crea y activa un entorno virtual por proyecto.
<code>IndentationError</code>	Mezcla de tabs y espacios. Usa 4 espacios de forma consistente.
El programa peta ante entradas raras	Falta manejo de excepciones. Envuelve E/S y parseo en <code>try/except</code> .

Preguntas frecuentes

 **¿Python 2 o 3?** Solo Python 3. Python 2 está fuera de soporte desde 2020; todo el material moderno usa 3.

 **¿Por qué str vs. bytes es tan importante en seguridad?** Porque sockets, criptografía y protocolos binarios trabajan con `bytes`. Confundirlos es la causa nº1 de errores en herramientas de red.

 **¿Necesito venv para todo?** Es una buena práctica: aísla dependencias, facilita reproducibilidad y evita romper herramientas del sistema que dependen de Python.

 **¿Cuándo usar `subprocess` en vez de librerías?** Para invocar herramientas externas (nmap, etc.). Cuando exista una librería nativa (requests, socket), préfiérela: es más segura y controlable.

Referencias

- Seitz & Arnold, *Black Hat Python* (No Starch Press).
- Documentación oficial de Python 3 — <https://docs.python.org/3/>
- Python `argparse` tutorial — <https://docs.python.org/3/howto/argparse.html>
- Real Python — <https://realpython.com/>

Siguiente clase

Clase 016 - Python para seguridad: sockets y programacion de red